Hibernate II

-By Miss. Geetanjali Gajbhar

Chapter 1

Association Mapping in Hibernate(Has-A relation)

- **OneToOne (Unidirectional and Bi-direction)**
- **OneToMany**
- **ManyToOne**
- **ManyToMany**

OneToOne (Unidirectional)

Example:

- **One Question can store/have only one answer.**

1. Create one Question Class Add @Entity And @Id

2. Create one Answer Class Add @Entity And @Id
3. We want **one-to-one mapping Unidirectional** hence add @OneToOne Annotation in Question class On Answer field.
4. Set the values and save the object using the session's save method.
5. Commit the Transactions.
6. Check the DB 2 tables will get created, and question table will have Foreign key of Answer table.

Extra Info: We have to map both the Entity classes in Hibernate.cfg.xml file.

Question Class

```
import javax.persistence.CascadeType;
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.OneToOne;

@Entity
public class Question {
    @Id
    private int question_id;
    private String question;
    @OneToOne(cascade = CascadeType.ALL)
    private Answer ans;


    public int getQuestion_id() {
        return question_id;
    }
    public void setQuestion_id(int question_id) {
        this.question_id = question_id;
    }
    public String getQuestion() {
```

```
        return question;
    }
    public void setQuestion(String question) {
        this.question = question;
    }
    public Answer getAns() {
        return ans;
    }
    public void setAns(Answer ans) {
        this.ans = ans;
    }
    public Question
        (int question_id, String question, Answer ans) {
        super();
        this.question_id = question_id;
        this.question = question;
        this.ans = ans;
    }
    public Question() {
        super();
        // TODO Auto-generated constructor stub
    }

}
```

Answer Class

```
package com.model;

import javax.persistence.Entity;
import javax.persistence.Id;

@Entity
public class Answer {
    @Id
```

```
    private int answer_id;
    private String answer;
    public int getAnswer_id() {
        return answer_id;
    }
    public void setAnswer_id(int answer_id) {
        this.answer_id = answer_id;
    }
    public String getAnswer() {
        return answer;
    }
    public void setAnswer(String answer) {
        this.answer = answer;
    }
    public Answer(int answer_id, String answer) {
        super();
        this.answer_id = answer_id;
        this.answer = answer;
    }
    public Answer() {
        super();
        // TODO Auto-generated constructor stub
    }

}
```

Test Class

```
package com.model;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;
```

```
public class Test1 {
    public static void main(String[] args) {

        //Configuration part
        //We need Session object
        Configuration cfg=
new Configuration().configure("hibernate.cfg.xml");

        SessionFactory sf = cfg.buildSessionFactory();
        Session session = sf.openSession();
        Transaction tx = session.beginTransaction();
        //------------------------------------------------------

        //Set the values
Answer ans1=new Answer(101,"java is OOPs Language");
Question que1=new Question(1001,"What is Java?", ans1);
        //------------------------------------------------------

    //save the object using Question object reference
        session.save(que1);
        tx.commit();
        System.out.println("Data inserted in Table");
        //------------------------------------------------------

        //close the session

        session.close();

    }

}
```

Check the console and check the Database.

Database table

Question table

question-id	question	ans_answer_id(Acts as a FK)
1001	What is java?	101

Answer Table

answer_id	answer
101	java is OOPs language

OneToOne (Bi-direction)

- Same as Above only change in Answer Class and Test class

Question Class

```
package com.model;

import javax.persistence.CascadeType;
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.OneToOne;
@Entity
public class Question {
    @Id
    private int question_id;
    private String question;
//-----------------------------------------------------
    @OneToOne(cascade = CascadeType.ALL)
    private Answer ans;
//-----------------------------------------------------

//generate getter setter and constructors
```

```
}
```

```
package com.model;

import javax.persistence.CascadeType;
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.OneToOne;

@Entity
public class Answer {
    @Id
    private int answer_id;
    private String answer;
//------------------------------
    @OneToOne(cascade = CascadeType.ALL)
    private Question que;

//------------------------------
//generate getter setter, Constructors


}
```

Test Class

```
package com.model;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;
```

```
public class Test1 {
    public static void main(String[] args) {

        //Configuration part
        //We need Session object
        Configuration cfg=
new Configuration().configure("hibernate.cfg.xml");
        SessionFactory sf = cfg.buildSessionFactory();
        Session session = sf.openSession();
        Transaction tx = session.beginTransaction();
        //-------------------------------------------------------

        //Set the values
        Question q=new Question();
        q.setQuestion_id(5001);
        q.setQuestion("What is Python?");

        Answer a=new Answer();
        a.setAnswer_id(1001);
        a.setAnswer("New Language");

        q.setAns(a);
        a.setQue(q);
        //-------------------------------------------------------

        //save using Question or ANswer object reference
        session.save(q);

        tx.commit();
        System.out.println("Data inserted in Table");
        //-------------------------------------------------------

        //close the session

        session.close();
```

```
    }

}
```

Check the console and check the Database.

Database table

Question Table

question_id	question	ans_answer_id (FK)
5001	What is Python?	1001

Answer Table

answer_id	answer	que_question_id (FK)
1001	New Language	5001

OneToMany and ManyToOne

One Question Can have Many Answers and Multiple Answers can have One Question

Question class

```
package com.model;

import java.util.List;

import javax.persistence.CascadeType;
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.OneToMany;
```

```
@Entity
public class Question {
    @Id
    private int question_id;
    private String question;
    @OneToMany(cascade = CascadeType.ALL)
    private List<Answer> ans;
    //-----------------------------------
//generate getter setter and constructors
}
```

Answer Class

```
package com.model;

import javax.persistence.CascadeType;
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.ManyToOne;
import javax.persistence.OneToOne;

import org.hibernate.annotations.ManyToAny;

@Entity
public class Answer {
    @Id
    private int answer_id;
    private String answer;
    @ManyToOne(cascade = CascadeType.ALL)//*****
    private Question que;


    //-----------------------------------
```

```
//generate getter setter and constructors
}
```

Test Class

```
package com.model;

import java.util.ArrayList;
import java.util.List;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;

public class Test1 {
    public static void main(String[] args) {

        //Configuration part
        //We need Session object
        Configuration cfg=
new Configuration().configure("hibernate.cfg.xml");
        SessionFactory sf = cfg.buildSessionFactory();
        Session session = sf.openSession();
        Transaction tx = session.beginTransaction();
        //-------------------------------------------------------
        //Set the values

        //Set Many Answers As it is one to many mapping
Answer ans1= new Answer(101,"Java Is OOPs Language");
Answer ans2= new Answer(102,"Java Is open source");
Answer ans3= new Answer(103,"Java Is secure");

        //Save the Answers in List
```

```
        List<Answer> list=new ArrayList<>();
        list.add(ans1);
        list.add(ans2);
        list.add(ans3);

Question que=new Question(5001,"What is Java?",list);

        //-------------------------------------------------------

        //save using Question or ANswer object reference
        session.save(que);

        tx.commit();
        System.out.println("Data inserted in Table");
        //-------------------------------------------------------

        //close the session

        session.close();

    }

}
```

Check the console and check the Database.

Database table: 3 tables will get created due to collection Property

Question Table

question_id	question
5001	What is Java?

Answer Table

answer_id	answer	question_id

101	Java Is OOPs Language	5001
102	Java Is open source	5001
103	Java Is secure	5001

Question_Answer Table

Question_que	ans_answer
5001	101
5001	102
5001	103

Note: If we don't want to create third table then use mappedBy property.

Question Class

```
@Entity
public class Question {
    @Id
    private int question_id;
    private String question;
    @OneToMany(cascade = CascadeType.ALL, mappedBy = "que")//***
    private List<Answer> ans;
```

Now only 2 tables will get created.

Question Table

question_id	question
5001	What is Java?

Answer Table

answer_id	answer	question_id

101	Java Is OOPs Language	5001
102	Java Is open source	5001
103	Java Is secure	5001

Many To Many

Many Questions Can have Many Answers and Many Answers can have Many Questions

Question Class

```
package com.model;

import java.util.List;

import javax.persistence.CascadeType;
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.ManyToMany;


@Entity
public class Question {
    @Id
    private int question_id;
    private String question;
    @ManyToMany(cascade = CascadeType.ALL)
    private List<Answer> ans;
    //-----------------------------------
    //generate getters setter and constructors

}
```

Answer Class

```
package com.model;

import java.util.List;

import javax.persistence.CascadeType;
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.ManyToMany;
import org.hibernate.annotations.ManyToAny;

@Entity
public class Answer {
    @Id
    private int answer_id;
    private String answer;
    @ManyToMany(cascade = CascadeType.ALL)
    private List<Question> que;
    //-----------------------------------
//generate getters setter and constructors
}
```

Test Class

```
package com.model;

import java.util.ArrayList;
import java.util.List;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;
```

```
public class Test1 {
    public static void main(String[] args) {

        //Configuration part
        //We need Session object
        Configuration cfg=
new Configuration().configure("hibernate.cfg.xml");
        SessionFactory sf = cfg.buildSessionFactory();
        Session session = sf.openSession();
        Transaction tx = session.beginTransaction();
        //-------------------------------------------------------
        //Set the values

        //Set Many Questions
        Question que1=new Question();
        que1.setQuestion_id(5001);
        que1.setQuestion("WHat is Java?");

        Question que2=new Question();
        que2.setQuestion_id(5002);
        que2.setQuestion("WHat is Python?");



        List<Question> lq=new ArrayList<>();
        lq.add(que1);
        lq.add(que2);

        //-----------------------------------------------
        //Set Many Answers As it is Many to many mapping

    Answer ans1=new Answer();
    ans1.setAnswer_id(101);
    ans1.setAnswer("oops language");
```

```
    Answer ans2=new Answer();
    ans2.setAnswer_id(102);
    ans2.setAnswer("Open source");

    List<Answer> la=new ArrayList<>();
    la.add(ans1);
    la.add(ans2);
    //-------------------------------------

    ans1.setQue(lq);
    que1.setAns(la);

    //-------------------------------------

        //save using Question or ANswer object reference
        session.save(que1);


        tx.commit();
        System.out.println("Data inserted in Table");
        //-------------------------------------------------------

        //close the session

        session.close();

    }

}
```

Check the console and check the Database.

4 Tables will get created Because of Collection Property.

Question Table

question_id	question
5001	What is Java?
5002	What is Python?

Answer table

answer_id	answer
101	OOPs language
102	Open Source

answer_question

answer_answer_id	question_question_id
101	5001
101	5002

question_answer

question_question_id	ans_answer_id
5001	101
5001	102

If We don't want these extra tables then use mappedBy property, Now 3 tables will get created.

Question Class

```
@Entity
public class Question {
    @Id
    private int question_id;
    private String question;
    @ManyToMany(cascade = CascadeType.ALL,mappedBy = "que")
    private List<Answer> ans;
```

Answer Class

```
@Entity
public class Answer {
    @Id
    private int answer_id;
    private String answer;
    @ManyToMany(cascade = CascadeType.ALL)
    private List<Question> que;
```

3 Tables will get created!

Chapter 2

Additional:

- **@Embeddable** : Class level annotation to inject that class into another class (Secondary Reference)
- **F.K.** → Foreign Key.→ Primary key of One table can act as Foreign key in another table.

How to Fetch List Data from DB?

- We have List of Questions in DB. Let us Fetch it!!

```
        //we have One question and many answers
   //so get the list of answers from one question

Question q = session.get(Question.class, 5001);
System.out.println("The question is"+q.getQuestion());
System.out.println("******************************");
```

```
List<Answer> ans = q.getAns();
for(Answer a:ans) {
    System.out.println(a.getAnswer());
}
```

- We must have to map **both the Entity** classes in Xml file.
- After every Program **do not forget** to delete the tables. Otherwise it will raise the Exception.
- **Cascading**: `cascade = CascadeType.ALL` What ever operation we perform on one entity, the same operation is performed on all related entities. Example: If I try to save Question object then related Answer's objects also get saved.

Hibernate Fetching Theory:

There are 2 Fetch Type

1. **Lazy** : Load the data only when we call the getter methods.
2. **Eager** : Load the entire data

Lazy loading or Eager Loading are the design patterns.

- By **default Hibernate provide Lazy loading**, If we Want to change it then,

```
@ManyToMany(cascade = CascadeType.ALL,
mappedBy = "que", fetch = FetchType.EAGER)
```

```
@ManyToMany(cascade = CascadeType.ALL,
mappedBy = "que", fetch = FetchType.LAZY)
```

Difference between save() and persist()

- **Save() and Persist() both methods are present in Session (I).**

sr.No	Save()	persist()
1	It stores object in database	It also stores object in database
2	It return generated id and return type is serializable	Return type void
3	It can save object within transaction boundaries and outside boundaries	It can only save object within the transaction boundaries
4	It is only supported by Hibernate	It is also supported by JPA

Chapter 3

1. **NamedQuery → for Single HQL query**
2. **NamedQueries → for Multiple HQL queries**
3. **NamedNativeQuery → for SQL query**
4. **NamedNativeQueries →for Multiple SQL queries**

NamedQuery:

It is way to use any query by some meaningful name.

Student.java

```
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.NamedQuery;

@NamedQuery(name="getAllData",
      query="from Student where rollNo=?")

@Entity

public class Student {
```

```
    @Id
    private int rollNo;
    public int getRollNo() {
        return rollNo;
    }
    public void setRollNo(int rollNo) {
        this.rollNo = rollNo;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    private String name;

}
```

Do not forget to add this mapping resource to Configuration file.

Test.Clas

```
package com.client;

import java.util.List;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.query.Query;

import com.config.HibernateUtil;
import com.model.Student;

public class Test {
```

```
    public static void main(String[] args) {


    SessionFactory sf=HibernateUtil.getSessionFactory();
    Session session =sf.openSession();
    Transaction tx=session.beginTransaction();
    //------------------------------------------------------------

Query<Student> query=session.getNamedQuery("getAllData");
    query.setParameter(0, 1);

    List<Student>list=query.getResultList();
    for(Student student:list) {
        System.out.println(student.getName());
        System.out.println(student.getRollNo());
    }

    }

}
```

NamedQueries:

@NameQueries annotation is used to define the multiple named queries

Student.class

```
import javax.persistence.Entity;

import javax.persistence.Id;
```

```
import javax.persistence.NamedQueries;
import javax.persistence.NamedQuery;

@NamedQueries({
@NamedQuery(name="getSingleData",
query="from Student where rollNo=?"),
@NamedQuery(name="getAllData",
query="from Student"),
@NamedQuery(name="deleteData",
query="delete from Student where rollno=?"),
@NamedQuery(name="updateData",
query="update Student set name=? where rollNo=?")
})

@Entity
public class Student {
    @Id
    private int rollNo;
    public int getRollNo() {
        return rollNo;
    }
    public void setRollNo(int rollNo) {
        this.rollNo = rollNo;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    private String name;

}
```

Test class

```
package com.client;

import java.util.List;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.query.Query;

import com.config.HibernateUtil;
import com.model.Student;

public class Test {

    public static void main(String[] args) {


    SessionFactory sf=HibernateUtil.getSessionFactory();
    Session session =sf.openSession();
    Transaction tx=null;

//-------------------------------------------------------------

System.out.println("******get single data*****");

Query<Student> query=session.
getNamedQuery("getSingleData");
    query.setParameter(0, 1);
    Student student=query.getSingleResult();

System.out.println(student.getName());
System.out.println(student.getRollNo());
//-------------------------------------------------------------
```

```
    System.out.println("****get all data*******");
    Query<Student> query1=session.
getNamedQuery("getAllData");
    List<Student>list=query1.getResultList();
    for(Student student1:list) {
        System.out.println(student1.getName());
        System.out.println(student1.getRollNo());

    }
//-------------------------------------------------------------

    System.out.println("*****update data****");
    tx=session.beginTransaction();
    Query<Student> query3=session.
getNamedQuery("updateData");
    query3.setParameter(0, "Sam");
    query3.setParameter(1, 1);
    query3.executeUpdate();
//-------------------------------------------------------------


    tx.commit();



/*  System.out.println("****delete data*******");
    tx=session.beginTransaction();
    Query<Student> query2=session.getNamedQuery("deleteData");
    query2.setParameter(0, 1);
    query2.executeUpdate();

    tx.commit();
    */

    }
```

```
}
```

Likewise we can use NamedNativeQuery And NamedNativeQueries for SQL Query.

Extra:

? → is called as Position Parameter